C++ and MLIR Fast Track

C++ 기본 개념, 실전 패턴, MLIR 감각까지 빠르게 익히는 교재

Python 개발자가 *C++/MLIR*에 빠르게 익숙해지기 위한 압축 학습서

목표

1. CMake와 빌드 흐름을 읽을 수 있다.
2. C++ 기본 개념을 설계 판단 수준으로 이해한다.
3. 자주 쓰는 코드 패턴을 통째로 외워 실전에 바로 적용한다.
4. MLIR 프로젝트 구조와 개발 흐름을 큰 그림으로 이해한다.

2026

Contents

Chapter 1

이 책을 읽는 방법

이 책의 목적은 C++ 전체를 완벽히 가르치는 것이 아니다. 대신 **C++ 코드가 왜 저렇게 생겼는지 이해하고**, **자주 나오는 패턴을 암기하고**, **MLIR 코드를 읽고 수정할 수 있는 상태**까지 빠르게 끌어올리는 것이다.

권장 학습 순서

1. Part I에서 CMake와 빌드 흐름을 먼저 익힌다.
2. Part II에서 소유권, 템플릿, virtual, move 같은 기본 개념을 잡는다.
3. Part III에서 실전 패턴을 암기한다.
4. Part IV에서 MLIR의 큰 그림, TableGen, 패스 개발 흐름을 연결한다.

Python 개발자 관점

Python에서는 런타임 모델이 대부분을 숨겨준다. C++에서는 타입, 수명, 소유권, 링크, 빌드 그래프를 직접 다뤄야 한다. 그래서 C++를 잘하려면 문법보다 먼저 **객체 수명**과 **타겟 관계**를 이해해야 한다.

Part I

빌드와 CMake

Chapter 2

MLIR 프로젝트를 위한 CMake 핵심

LLVM/MLIR을 다루려면 먼저 `cmake`가 무엇을 하는지 감이 잡혀야 한다. 핵심은 **소스 파일 나열**이 아니라 **타겟 그래프 설계**다.

2.1 전체 흐름

LLVM/MLIR 빌드의 기본 흐름

```
git clone https://github.com/llvm/llvm-project.git
cd llvm-project
git checkout llvmorg-18.1.0

cmake -S llvm -B build \
  -DCMAKE_BUILD_TYPE=Release \
  -DCMAKE_INSTALL_PREFIX=$HOME/llvm-install \
  -DLLVM_ENABLE_PROJECTS=mlir \
  -DLLVM_TARGETS_TO_BUILD=host \
  -DLLVM_ENABLE_ASSERTIONS=ON

cmake --build build -j$(sysctl -n hw.ncpu)
cmake --install build
```

배운 것 정리

- `-S`는 소스 루트, `-B`는 빌드 디렉토리다.
- `configure` 단계와 `build` 단계는 다르다.
- MLIR 개발에서는 `Release + Assertions` 조합이 실용적이다.
- LLVM은 변동이 빠르므로 `main`보다 릴리스 태그 고정이 안전하다.

2.2 CMake가 실제로 하는 일

기본 흐름

```
cmake -S . -B build          # configure: 빌드 그래프 생성
cmake --build build          # build: 실제 컴파일링크/
ctest --test-dir build       # test: 테스트 실행
cmake --install build        # install: 산출물 배치
```

핵심 시야

- CMake는 컴파일러가 아니다. build.ninja, Makefile, Xcode project 같은 빌드 시스템을 생성한다.
- **configure 시점**과 **build 시점**을 구분해야 한다.
- CMake 파일을 읽을 때는 파일 목록보다 "무슨 타겟이 생기고, 무슨 속성이 붙는가"를 먼저 본다.

Chapter 3

반드시 알아야 할 CMake 명령어

3.1 먼저: CMake를 어떻게 읽어야 하나

엄밀하게 말하면 CMake는 C++처럼 강한 의미의 예약어 목록이 있는 언어가 아니다. 실전에서 중요한 것은 아래 세 가지를 구분하는 것이다.

1. **명령**: `project()`, `add_library()`, `find_package()`처럼 호출하는 것
2. **키워드성 인자**: `PUBLIC`, `PRIVATE`, `REQUIRED`, `CONFIG`, `VERSION`처럼 명령의 의미를 바꾸는 것
3. **변수**: `CMAKE_CXX_STANDARD`, `PROJECT_SOURCE_DIR`, `BUILD_TESTING`처럼 상태를 담는 것

CMake 파일을 읽을 때 항상 할 질문

1. 지금 이 줄은 **타겟을 생성하는가**, 아니면 **기존 타겟을 수정하는가**?
2. configure 시점에 처리되는가, 아니면 build 시점 산출물에 영향을 주는가?
3. 현재 타겟에만 적용되는가, 소비자에게도 전파되는가?
4. 변수 조작인가, 타겟 속성 조작인가?

3.2 최소 예제를 먼저 해석해 보기

가장 작은 예제

```
cmake_minimum_required(VERSION 3.20)
project(kv_cli VERSION 0.1.0 LANGUAGES CXX)

add_executable(kv
  src/main.cpp
)
```

줄	의미
`cmake_minimum_required(VERSION 3.20)`	최소 CMake 버전과 정책 기준선을 정한다. 너무 낮은 버전이면 바로 실패시킨다.
`project(kv_cli VERSION 0.1.0 LANGUAGES CXX)`	프로젝트 이름, 버전, 활성 언어를 선언한다. 이 줄이 지나야 C++ 컴파일러 탐색과 관련 변수 초기화가 제대로 이뤄진다.

줄	의미
`add_executable(kv src/main.cpp)`	`kv`라는 실행 파일 타겟을 만들고 그 소스로 `src/main.cpp`를 연결한다. 핵심은 파일보다 타겟이 생긴다는 점이다.

Python 개발자 관점

Python의 `pyproject.toml`은 패키지 메타데이터 중심이다. CMake는 더 낮은 레벨에서 컴파일러, 링크, 설치, 타겟 전파 규칙을 직접 모델링한다.

3.3 레벨 1: 검색 없이 써야 하는 필수 명령

아래 명령어들은 "C++ 개발자에게 CMake를 더 요구하진 않겠다"고 말할 수 있는 최소 핵심이다. 이름만 아는 것으로는 부족하고, **대표 예제와 자주 쓰는 인자의 의미**까지 같이 떠올라야 한다.

`cmake_minimum_required(VERSION ...)`

```
cmake_minimum_required(VERSION 3.20)
```

- **역할**: 이 프로젝트가 기대하는 최소 CMake 버전과 정책 기준을 정한다.
- **반드시 알아둘 인자**: `VERSION`은 최소 버전 선언이다.
- **언제 쓰나**: 루트 `CMakeLists.txt` 최상단.
- **실전 판단**: 너무 낮게 적으면 실제로 쓰는 문법과 안 맞고, 너무 높게 적으면 불필요하게 오래된 환경을 끊는다.

`project(...)`

```
project(kv_cli VERSION 0.1.0 LANGUAGES CXX)
```

인자	의미
`kv_cli`	프로젝트 이름. `PROJECT_NAME` 등에 반영된다.
`VERSION 0.1.0`	프로젝트 버전 메타데이터. 설치, configure 파일, 문서에 재사용 가능하다.
`LANGUAGES CXX`	C++ 언어만 활성화. 필요하면 `C`, `CUDA` 등을 추가한다.

- **역할**: 프로젝트 이름을 붙이고, 언어를 켜는 트리거다.
- **언제 쓰나**: 보통 루트에서 한 번.
- **실전 판단**: 안 쓰는 언어를 켜지 않는 것이 configure를 단순하게 만든다.

`add_executable(...)`

```
add_executable(kv_cli
  src/main.cpp
```

```
)
```

- **역할**: 실행 파일 타겟 생성
- **대표 인자**: 타겟 이름, 소스 목록
- **언제 쓰나**: 최종 바이너리 정의
- **실전 판단**: 실행 파일은 "소비자"이므로 다른 라이브러리에 의존하는 쪽이다.

add_library(...)

```
add_library(kv_core
  src/store.cpp
  src/command.cpp
)
```

- **역할**: 재사용 가능한 라이브러리 타겟 생성
- **대표 인자**: 타겟 이름, 소스 목록
- **자주 보는 변형**: `STATIC`, `SHARED`, `INTERFACE`
- **실전 판단**: 큰 프로젝트는 실행 파일보다 라이브러리 단위 설계가 중요하다.

주의

`INTERFACE` 라이브러리는 실제 바이너리를 만들지 않는다. header-only 라이브러리나 사용 규칙 묶음을 표현할 때 쓴다.

target_link_libraries(...)

```
target_link_libraries(kv_cli PRIVATE kv_core)
```

인자	의미
`PRIVATE`	현재 타겟에만 적용. 소비자에게 전파 안 함
`PUBLIC`	현재 타겟도 사용하고 소비자에게도 전파
`INTERFACE`	현재 타겟 자체는 안 쓰고 소비자에게만 전파

- **역할**: 링크 관계를 만들고, usage requirement 전파를 정의한다.
- **언제 쓰나**: 라이브러리 의존성 선언
- **실전 판단**: 실행 파일에서는 대체로 `PRIVATE`부터 시작하면 된다.

target_include_directories(...)

```
target_include_directories(kv_core
  PUBLIC include
  PRIVATE src
)
```

- **역할**: include path를 타겟 단위로 설정
- **자주 쓰는 인자**: `PUBLIC`, `PRIVATE`, `INTERFACE`
- **언제 쓰나**: 공개 헤더와 비공개 헤더 경로를 분리할 때
- **실전 판단**: 공개 API 헤더가 있는 곳은 `PUBLIC`, 구현 상세용 헤더는 `PRIVATE`

`target_compile_features(...)`

```
target_compile_features(kv_core PUBLIC cxx_std_20)
```

- **역할**: 요구하는 언어 기능 수준을 타겟 단위로 선언
- **대표 인자**: `cxx_std_17`, `cxx_std_20`
- **언제 쓰나**: 전역 `CMAKE_CXX_STANDARD` 대신 타겟 중심으로 제어할 때
- **실전 판단**: 라이브러리 공개 헤더가 특정 표준 기능을 요구하면 `PUBLIC`

`target_compile_definitions(...)`

```
target_compile_definitions(kv_core
  PRIVATE KV_ENABLE_LOGGING=1
)
```

- **역할**: 컴파일 정의(`-D...`)를 타겟에 붙인다.
- **언제 쓰나**: 빌드 설정에 따라 조건부 컴파일이 필요할 때
- **실전 판단**: 전역 `add_definitions()`보다 이쪽이 훨씬 안전하다.

`target_sources(...)`

```
target_sources(kv_core
  PRIVATE
    src/cache.cpp
    src/index.cpp
)
```

- **역할**: 이미 만든 타겟에 소스를 추가
- **언제 쓰나**: 타겟 선언과 소스 관리를 분리하고 싶을 때, 하위 디렉토리에서 소스를 덧붙일 때
- **실전 판단**: 복잡한 프로젝트에서 `add_library(...)` 한 줄이 과하게 길어지는 것을 줄일 수 있다.

`find_package(...)`

MLIR 프로젝트 최소 골격

```
find_package(LLVM REQUIRED CONFIG)
find_package(MLIR REQUIRED CONFIG)

list(APPEND CMAKE_MODULE_PATH "${LLVM_CMAKE_DIR}")
list(APPEND CMAKE_MODULE_PATH "${MLIR_CMAKE_DIR}")
```

```

include(AddLLVM)
include(AddMLIR)

add_executable(my-opt tools/my-opt.cpp)
target_link_libraries(my-opt PRIVATE MLIRIR MLIRParser MLIRPass)
```

인자	의미
LLVM, MLIR	패키지 이름
REQUIRED	못 찾으면 configure 실패
CONFIG	패키지가 제공하는 XXXConfig.cmake를 찾겠다는 뜻

- **역할**: 외부 패키지의 경로, imported target, 보조 모듈 정보를 현재 프로젝트로 가져온다.
- **언제 쓰나**: fmt, spdlog, LLVM, MLIR 같은 외부 생태계를 연결할 때
- **실전 판단**: 현대 CMake에서는 Config mode + imported target 사용이 기본이다.

Python 개발자 관점

find_package(MLIR)는 Python의 import mlir와 완전히 같지는 않지만, 감각적으로는 비슷하다. MLIR이 어디 설치되어 있고 어떤 include/lib/cmake 파일을 써야 하는지 현재 프로젝트로 가져오는 단계다.

add_subdirectory(...)

```
add_subdirectory(lib)
add_subdirectory(tools)
add_subdirectory(tests)
```

- **역할**: 하위 디렉토리의 CMakeLists.txt를 현재 빌드 그래프에 포함
- **언제 쓰나**: 프로젝트 구조를 디렉토리별로 나눌 때
- **실전 판단**: 루트에 모든 타겟 정의를 몰아넣지 말고, 모듈별로 분리한다.

option(...)

```
option(KV_BUILD_TESTS "Build tests" ON)
```

- **역할**: 사용자가 -D...=ON/OFF로 바꿀 수 있는 boolean 옵션 생성
- **인자**: 변수 이름, 설명 문자열, 기본값
- **언제 쓰나**: 테스트, 예제, 벤치마크, 실험 기능 on/off
- **실전 판단**: 사용자가 꺼야 할 수 있는 기능이면 set()보다 option()이 낫다.

include(...)

```
include(AddLLVM)
include(AddMLIR)
```

- **역할**: 다른 CMake 모듈 파일을 로드
- **언제 쓰나**: 생태계 전용 매크로를 가져올 때
- **실전 판단**: LLVM/MLIR에서는 `AddLLVM.cmake`, `AddMLIR.cmake`를 자주 본다.

install(...)

```
install(TARGETS kv_core kv_cli
  RUNTIME DESTINATION bin
  LIBRARY DESTINATION lib
  ARCHIVE DESTINATION lib
)
```

인자	의미
TARGETS	설치할 타겟 목록
RUNTIME DESTINATION	실행 파일 설치 경로
LIBRARY DESTINATION	공유 라이브러리 설치 경로
ARCHIVE DESTINATION	정적 라이브러리 설치 경로

- **역할**: 산출물을 다른 프로젝트가 소비할 수 있는 형태로 배치
- **언제 쓰나**: 로컬 개발을 넘어서 재사용/배포를 고려할 때
- **실전 판단**: "내 PC에서만 돌아간다"에서 "다른 프로젝트가 쓸 수 있다"로 가는 경계다.

enable_testing() / add_test(...)

```
enable_testing()
add_test(NAME store_test COMMAND store_test)
```

- **역할**: CTest 기반 테스트 등록
- **언제 쓰나**: `ctest`로 실행 가능한 테스트를 구성할 때
- **실전 판단**: 단순히 실행 파일을 만드는 것만으로는 테스트가 아니다. `add_test()`까지 있어야 테스트 그래프에 들어간다.

configure_file(...)

```
configure_file(
  ${CMAKE_CURRENT_SOURCE_DIR}/config.h.in
  ${CMAKE_CURRENT_BINARY_DIR}/generated/config.h
  @ONLY
)
```

- **역할**: configure 시점에 템플릿 파일을 치환해 생성
- **언제 쓰나**: 버전, 경로, 옵션 값을 헤더나 설정 파일로 주입할 때
- **실전 판단**: build 때마다 도는 codegen이 아니라 configure 단계 생성물이라는 점이 중요하다.

3.4 옵션과 키워드를 읽는 감각

단어	주로 나오는 곳	실전 해석
`VERSION`	`cmake_minimum_required`, `project`	최소 도구 버전인지, 프로젝트 버전인지 문맥으로 구분
`REQUIRED`	`find_package`	없으면 진행하지 말고 실패
`CONFIG`	`find_package`	Config mode 우선
`PUBLIC`	`target_*` 계열	현재 타겟과 소비자 모두에 적용
`PRIVATE`	`target_*` 계열	현재 타겟에만 적용
`INTERFACE`	`target_*` 계열	소비자에게만 적용

가장 중요한 세 단어

`PUBLIC`, `PRIVATE`, `INTERFACE`는 단순 옵션이 아니라 CMake 설계의 핵심이다. 이 세 단어의 감각이 없으면, 프로젝트가 커질수록 include path와 링크 전파가 왜 그렇게 되는지 설명할 수 없게 된다.

Chapter 4

타겟 중심 사고와 의존성 전파

4.1 파일보다 타겟이 주인공이다

좋은 방향의 CMake

```
add_library(kv_core
  src/store.cpp
  src/command.cpp
)

target_include_directories(kv_core
  PUBLIC ${CMAKE_CURRENT_SOURCE_DIR}/include
)

target_compile_features(kv_core PUBLIC cxx_std_20)

add_executable(kv_cli src/main.cpp)
target_link_libraries(kv_cli PRIVATE kv_core)
```

배운 것 정리

- `add_library`, `add_executable`는 타겟을 만든다.
- `target_*` 명령은 그 타겟에 속성을 붙인다.
- 좋은 CMake는 "파일 모음"이 아니라 "타겟 관계도"로 읽힌다.

4.2 `PUBLIC` / `PRIVATE` / `INTERFACE` 실제 해석

```
target_include_directories(kv_core
  PUBLIC include
  PRIVATE src
)

target_link_libraries(kv_cli PRIVATE kv_core)
```

- `include`는 `kv_core` 공개 헤더 위치이므로 소비자도 알아야 한다.
- `src`는 내부 구현 전용이라 소비자에게 줄 필요가 없다.
- `kv_cli`는 실행 파일이므로 보통 소비자에게 아무 것도 전파할 필요가 없다.

주의

`PUBLIC`을 대충 남발하면 의존성이 과하게 전파되고, `PRIVATE`를 남발하면 소비자 쪽에서 헤더를 못 찾거나 링크가 깨진다. 공개 API가 실제로 무엇을 노출하는지 기준으로 결정해야 한다.

4.3 의존성 전파 예시

```
add_library(kv_storage src/store.cpp)
add_library(kv_commands src/command.cpp)

target_link_libraries(kv_commands PUBLIC kv_storage)

add_executable(kv_cli src/main.cpp)
target_link_libraries(kv_cli PRIVATE kv_commands)
```

위 경우 `kv_cli`는 `kv_storage`를 직접 링크하지 않아도, `kv_commands`의 `PUBLIC` 전파 덕분에 필요한 정보를 전달받는다.

Chapter 5

외부 패키지, 테스트, 설치까지 연결하기

5.1 find_package와 imported target

```
find_package(fmt REQUIRED CONFIG)
find_package(spdlog REQUIRED CONFIG)

target_link_libraries(kv_core
  PUBLIC
    fmt::fmt
    spdlog::spdlog
)
```

- 패키지가 제공하는 imported target은 include path, 링크 옵션, 컴파일 정의를 함께 묶어준다.
- 절대 경로를 직접 박는 방식보다 imported target이 훨씬 안전하다.
- 현대 CMake에서는 "경로를 손으로 짜맞추기"보다 "패키지가 제공한 타겟을 링크하기"가 기본이다.

주의

`include_directories(/usr/local/include)`와 `link_directories(...)`를 전역으로 뿌리는 방식은 현대 CMake에서 거의 항상 열세다.

5.2 테스트 연결

```
include(CTest)

if(BUILD_TESTING)
  add_executable(store_test tests/store_test.cpp)
  target_link_libraries(store_test PRIVATE kv_core)
  add_test(NAME store_test COMMAND store_test)
endif()
```

- `include(CTest)` 또는 `enable_testing()`로 테스트 인프라를 켠다.
- 테스트 실행 파일을 만들고, `add_test()`로 등록한다.

- `ctest --test-dir build --output-on-failure`로 실행한다.

5.3 설치와 재사용

```
install(TARGETS kv_core kv_cli
  EXPORT kvTargets
  RUNTIME DESTINATION bin
  LIBRARY DESTINATION lib
  ARCHIVE DESTINATION lib
)

install(DIRECTORY include/ DESTINATION include)

install(EXPORT kvTargets
  NAMESPACE kv::
  DESTINATION lib/cmake/kv
)
```

- `install(TARGETS ...)`는 바이너리 산출물 배치
- `install(DIRECTORY ...)`는 헤더/리소스 배치
- `install(EXPORT ...)`는 다른 프로젝트가 `find_package(kv)`로 소비할 수 있는 메타데이터 생성

Python 개발자 관점

Python 패키지를 wheel로 배포해 다른 프로젝트가 import하는 흐름과 비슷하지만, C++은 ABI, 헤더, 링크 정보까지 함께 전달해야 한다.

5.4 configure 시점 파일 생성

```
configure_file(
  ${CMAKE_CURRENT_SOURCE_DIR}/config.h.in
  ${CMAKE_CURRENT_BINARY_DIR}/generated/config.h
  @ONLY
)
```

config.h.in

```
#pragma once

#define KV_VERSION "@PROJECT_VERSION@"
#define KV_ENABLE_CACHE @KV_ENABLE_CACHE@
```

- `configure_file()`는 configure 시점 치환이다.
- build 때마다 재실행되는 codegen과는 역할이 다르다.
- version/header/config template 주입에 적합하다.

Chapter 6

팀 개발에서 필요한 CMake 감각

6.1 Presets와 Toolchain

개념 예시, 실제 파일은 JSON

```
{
  "version": 6,
  "configurePresets": [
    {
      "name": "dev",
      "generator": "Ninja",
      "binaryDir": "${sourceDir}/build/dev",
      "cacheVariables": {
        "CMAKE_BUILD_TYPE": "Debug",
        "KV_BUILD_TESTS": "ON"
      }
    }
  ]
}
```

- Presets는 팀이 같은 configure 인자를 재현하게 해준다.
- Toolchain 파일은 cross compile, 특정 SDK, sysroot, 전용 컴파일러 조합을 고정할 때 쓴다.
- README에 긴 명령어를 적는 것보다 설정 파일로 고정하는 편이 팀 생산성이 높다.

6.2 자주 나는 에러와 먼저 볼 곳

6.3 실전에서 자주 나는 에러

증상	원인 후보	먼저 볼 곳
`undefined reference`	링크 라이브러리 누락	`target_link_libraries`
`file not found`	include 경로 누락	`target_include_directories`, `find_package`

증상	원인 후보	먼저 볼 곳
`No rule to make target`	타겟/생성 파일 연결 누락	`add_subdirectory`, generated sources
`RTTI mismatch`	컴파일 옵션 불일치	LLVM/MLIR 빌드 옵션, `-fno-rtti` 여부

6.4 피해야 할 안티패턴

- 루트에서 `include_directories()`를 전역 살포
- 패키지 대신 절대 경로 하드코딩
- `file(GLOB ...)`로 소스 자동 수집 후 변경 추적 불명확하게 만들기
- `PUBLIC`/`PRIVATE`를 감으로만 결정
- `set()`로 모든 설정을 해결하려고 하기

실전 체크리스트

- 소스와 빌드를 분리하는 out-of-source build를 유지할 것
- 실행파일보다 라이브러리 단위로 구조를 나눌 것
- MLIR을 찾지 못하면 먼저 `MLIR_DIR`과 `LLVM_DIR`를 의심할 것
- 에러가 링크 단계인지 컴파일 단계인지 먼저 분리할 것
- 타겟 생성 명령과 타겟 수정 명령을 머릿속에서 구분할 것

Chapter 7

MLIR 프로젝트에서 CMake를 읽는 눈

7.1 전형적인 MLIR 프로젝트 골격

MLIR 도구 프로젝트의 최소 뼈대

```
cmake_minimum_required(VERSION 3.20)
project(MyMlirProject LANGUAGES CXX)

find_package(LLVM REQUIRED CONFIG)
find_package(MLIR REQUIRED CONFIG)

list(APPEND CMAKE_MODULE_PATH "${LLVM_CMAKE_DIR}")
list(APPEND CMAKE_MODULE_PATH "${MLIR_CMAKE_DIR}")
include(AddLLVM)
include(AddMLIR)

add_subdirectory(lib)
add_subdirectory(tools)
```

도구 타겟 예시

```
add_executable(my-opt my-opt.cpp)
target_link_libraries(my-opt
  PRIVATE
    MyDialect
    MyConversion
    MLIRIR
    MLIRParser
    MLIRPass
)
```

MLIR CMake를 읽을 때 먼저 볼 것

1. `find_package(LLVM/MLIR)`가 어디서 설정을 가져오는가
2. `include(AddLLVM)`, `include(AddMLIR)`로 어떤 매크로를 쓰는가
3. dialect / conversion / tools가 어떤 라이브러리 계층으로 분리돼 있는가
4. RTTI, assertions, generated headers 같은 상위 생태계 제약이 무엇인가

Quiz

Q1. `target_link_libraries(kv_cli PRIVATE kv_core)`를 한국어로 정확히 해석해 보라.
정답: `kv_cli` 실행 파일은 `kv_core` 라이브러리에 의존하며, 그 관계는 현재 타겟에만 적용된다.

Quiz

Q2. `find_package(fmt REQUIRED CONFIG)`에서 `REQUIRED`를 빼면 어떻게 되나?
정답: 패키지를 못 찾았을 때 즉시 실패하지 않고, 이후 조건 분기나 별도 처리를 할 여지가 생긴다.

Quiz

Q3. 지금 당장 검색 없이 떠올라야 할 명령 목록은?
정답: `cmake_minimum_required`, `project`, `add_library`, `add_executable`, `target_link_libraries`, `target_include_directories`, `target_compile_features`, `find_package`, `add_subdirectory`, `option`, `install`, `add_test`, `configure_file`.

Part II

C++ 기본 개념

Chapter 8

C++를 읽을 때 가장 먼저 봐야 하는 것

C++ 코드를 읽을 때 변수 타입보다 먼저 봐야 하는 것은 **누가 소유하는가**, **언제 파괴되는가**, **런타임 디스패치인가 컴파일 타임 디스패치인가**다.

이 파트의 목표

여기서는 문법 목록을 외우는 대신, 설계 판단 기준을 익힌다. 즉 "왜 이 타입을 썼는가", "왜 virtual이 아니라 template인가", "왜 값으로 안 받고 참조로 받는가" 같은 질문에 답할 수 있어야 한다.

8.1 Template vs Virtual

한 줄 요약

- **Virtual**: 런타임에 어떤 자식이 올지 모를 때
- **Template**: 컴파일 타임에 타입이 이미 정해져 있을 때

Virtual: 런타임 디스패치

```cpp
class Serializer {
public:
    virtual ~Serializer() = default;
    virtual std::string name() const = 0;
};

class JsonSerializer : public Serializer {
public:
    std::string name() const override { return "json"; }
};

void use(Serializer &s) {
    std::cout << s.name() << "\n";
}
```

Template: 컴파일 타임 디스패치

```
template <typename T>
void dump(const T &value) {
    std::cout << value.toString() << "\n";
}
```

배운 것 정리

- factory가 부모 포인터를 반환하면 virtual일 가능성이 높다.
- `OpConversionPattern<OpT>`처럼 타입이 템플릿 인자로 박혀 있으면 template 쪽이다.
- 성능보다 설계 이유를 먼저 본다. 런타임 선택이 필요하면 virtual이 맞다.

virtual과 template의 차이는 단순히 "느리냐 빠르냐"가 아니다. 핵심은 **누가 선택권을 가지는가**다. virtual은 호출 시점까지 구체 타입 결정을 미룬다. 템플릿은 작성 시점에 타입을 박아 넣고 그 대가로 최적화 기회를 얻는다. MLIR에서 두 방식이 자주 섞이는 이유도 여기에 있다. 예를 들어 `OpConversionPattern<MyOp>`는 "어떤 op를 처리하는가"는 컴파일 타임에 고정하지만, 실제로 어느 패턴이 실행될지는 프레임워크가 런타임에 고른다.

8.2 포인터와 참조: 소유권 관점으로 보기

형태	소유권	언제 쓰나
T*	없음	null 가능, 빌려서 잠깐 접근
T\&	없음	null이 되면 안 되는 별명
`std::unique_ptr<T>`	단독 소유	수명 관리가 필요한 힙 객체
`std::shared_ptr<T>`	공유 소유	정말 여러 주체가 함께 오래 살아야 할 때

Python 개발자 관점

Python에서는 이름이 객체를 참조하고 GC가 나중에 정리한다. C++에서는 **누가 해제 책임을 갖는지**를 코드 구조에 드러내야 한다. 그래서 `unique_ptr`이 자주 나오고, raw pointer는 보통 "내 것이 아니다"라는 신호다.

이 표를 볼 때 가장 중요한 질문은 "무엇이 더 편한가"가 아니라 "무엇이 소유권을 가장 정직하게 드러내는가"다. C++ 코드가 읽기 어려워지는 가장 흔한 이유는 로직이 아니라 소유권이 숨겨져 있기 때문이다. MLIR/LLVM 쪽 코드가 raw pointer와 reference를 많이 쓰는 것도 "이 객체를 내가 파괴하지 않는다"는 사실을 드러내기 위해서다.

8.3 Move Semantics

std::move의 의미

```
std::string name = "alice";
```

```
std::vector<std::string> names;

names.push_back(name);            // 복사
names.push_back(std::move(name)); // 이동
```

중요한 오해 바로잡기

`std::move`는 실제로 이동하지 않는다. "이 객체를 rvalue로 취급해도 된다"고 표시하는 캐스트에 가깝다. 실제 이동은 이동 생성자나 이동 대입 연산자가 수행한다.

이 개념이 중요한 이유는 "성능 최적화 트릭"이 아니라 **소유권 이전의 문법**이기 때문이다. `RewritePatternSet`을 `std::move`해서 넘길 때의 의미는 "복사 비용을 줄이자"보다 "이 컬렉션의 이후 생명주기는 프레임워크가 책임진다"에 더 가깝다.

8.4 Value Semantics vs Reference Semantics

복사를 피하는 기본 습관

```
void process(const std::vector<int> &data); // 읽기 전용, 복사 없음
void mutate(std::vector<int> &data);        // 수정 가능
void consume(std::vector<int> data);        // 값으로 받으면 소유권 복사
```

배운 것 정리

- 큰 객체를 읽기만 하면 `const T\&`
- 수정해야 하면 `T\&`
- 소유권을 넘기거나 복사본이 필요하면 값 전달
- 작은 값 타입은 그냥 값으로 받아도 된다

값 의미론(value semantics)이 좋은 이유는 예측 가능성 때문이다. 값으로 복사하면 서로 영향을 끊을 수 있고, 참조로 넘기면 같은 객체를 함께 본다. C++에서는 이 차이가 성능에도 직접 영향을 주므로, 설계와 성능이 분리되지 않는다. 그래서 함수 시그니처를 읽는 것만으로도 그 함수의 의도를 상당 부분 알 수 있다.

8.5 LLVM 스타일 타입 시스템

LLVM/MLIR 코드에서는 C++ RTTI 대신 `isa`, `cast`, `dyn_cast`를 많이 쓴다.

LLVM 캐스팅 3형제

```
if (isa<mlir::FuncOp>(op)) {
    auto func = cast<mlir::FuncOp>(op);
}

if (auto func = dyn_cast<mlir::FuncOp>(op)) {
    // 성공했을 때만 func 사용
}
```

주의

`cast<T>`는 실패하면 죽는다. 성공이 확실할 때만 쓴다. 확실하지 않으면 `dyn_cast<T>`부터 쓴다.

이 섹션이 중요한 이유는 MLIR 코드를 읽을 때 `dynamic_cast`가 거의 안 보이기 때문이다. LLVM은 성능과 일관성을 위해 자체 캐스팅 규약을 강하게 밀고 있고, 이 규약을 익혀 두면 "이 코드는 실패 가능성을 어떻게 표현하는가"까지 함께 읽을 수 있다.

Chapter 9

Python 개발자가 헷갈리기 쉬운 C++ 지점

9.1 헤더와 구현 분리

헤더 파일

```
#ifndef LOGGER_H
#define LOGGER_H

#include <string>

class Logger {
public:
    void info(const std::string &msg);
};

#endif
```

구현 파일

```
#include "Logger.h"
#include <iostream>

void Logger::info(const std::string &msg) {
    std::cout << msg << "\n";
}
```

Python 개발자 관점

Python은 파일 자체가 모듈이다. C++은 선언과 정의를 분리하면서 컴파일 시간을 줄이고, 여러 번 포함되는 상황을 제어한다.

9.2 Forward Declaration

가능한 경우 전방 선언

```
class Engine;

class Car {
public:
    void setEngine(Engine *engine);
private:
    Engine *engine_ = nullptr;
};
```

배운 것 정리

- 포인터/참조만 들고 있으면 전방 선언으로 충분한 경우가 많다.
- 객체 자체를 멤버로 들고 있으면 완전한 정의가 필요하다.
- 헤더 의존성을 줄이면 전체 빌드 시간이 줄어든다.

9.3 CLI 도구의 기본 골격

LLVM 스타일 CLI 도구

```
#include "llvm/Support/CommandLine.h"
#include "llvm/Support/InitLLVM.h"

llvm::cl::opt<std::string> inputFile(
    llvm::cl::Positional,
    llvm::cl::desc("<input file>"),
    llvm::cl::Required);

int main(int argc, char **argv) {
    llvm::InitLLVM y(argc, argv);
    llvm::cl::ParseCommandLineOptions(argc, argv);
    return 0;
}
```

이 패턴은 단순한 main 함수 예제가 아니라 “LLVM 생태계에 들어갈 때의 표준 입구”에 가깝다. 명령행 파싱, 초기화, 에러 출력, 종료 처리 방식이 일반 C++ 툴과 조금 다르기 때문에, MLIR 도구를 처음 만들 때는 이 모양을 통째로 복사해서 시작하는 편이 낫다.

Part III

암기할 가치가 있는 C++ 패턴

Chapter 10

매일 보게 되는 패턴

이 파트의 목표는 레퍼런스 문서를 대체하는 것이 아니라, **실전에서 자주 반복되는 구조를 통째로 기억하게 하는 것**이다. 패턴을 외운다는 것은 문장을 외운다는 뜻이 아니라 "이 문제가 보이면 이 구조가 떠오른다"는 상태를 만드는 것이다.

10.1 RAII

자원 획득과 해제를 객체 수명에 묶기

```
{
    std::ifstream file("data.txt");
    std::string line;
    while (std::getline(file, line)) {
        // 사용
    }
} // file 소멸자에서 자동 close
```

핵심

RAII는 "자원을 얻자마자 관리 객체 안에 넣고, 해제는 소멸자에 맡긴다"는 습관이다. C++에서 `finally` 대신 소멸자가 중요한 이유다.

이 패턴이 교재 맨 앞쪽에 반복해서 나오는 이유는, C++의 절반이 사실상 수명 관리 문제이기 때문이다. 파일, 락, 메모리, 임시 상태 복원, 삽입 지점 관리처럼 "반드시 되돌려야 하는 것"은 거의 모두 RAII로 귀결된다. RAII가 익숙해지면 예외나 중간 `return` 때문에 정리 코드가 빠질 걱정이 크게 줄어든다.

10.2 Rule of Zero

가능하면 직접 소멸자를 쓰지 않는다

```
struct Config {
    std::string name;
    std::vector<int> values;
```

```
    std::unique_ptr<int> handle;
};
```

배운 것 정리

멤버가 이미 자기 자원을 잘 관리한다면, 생성자/소멸자/복사/이동을 굳이 직접 만들지 않는 편이 더 안전하다.

초보자가 가장 자주 하는 실수는 raw pointer를 직접 들고 있으면서 소멸자만 겨우 추가하는 것이다. 그러면 곧바로 복사 생성자, 복사 대입, 이동 생성자, 이동 대입까지 전부 고민해야 한다. 반대로 멤버가 `string`, `vector`, `unique_ptr` 같은 RAII 타입이면, 컴파일러가 기본 멤버 함수를 거의 항상 더 잘 만들어 준다.

10.3 Factory 함수

생성과 사용 분리

```
class Optimizer {
public:
    virtual ~Optimizer() = default;
    virtual void step() = 0;
};

std::unique_ptr<Optimizer> createOptimizer(const Config &config) {
    if (config.type == "adam")
        return std::make_unique<Adam>(config.lr);
    return std::make_unique<SGD>(config.lr);
}
```

Factory는 "생성 로직을 한 곳에 모으는 패턴"이고, virtual interface는 "사용 코드는 구체 타입을 몰라도 되게 하는 패턴"이다. 둘을 같이 쓰면 생성 시점에는 타입을 고르되, 사용 시점에는 추상화 뒤에 숨길 수 있다. MLIR에서도 `create...Pass()` 류 함수가 이런 구조를 많이 따른다.

10.4 const 참조 매개변수

복사 없이 읽기

```
void process(const std::vector<int> &data);
void modify(std::vector<int> &data);
void check(int value);
```

큰 객체를 읽기만 할 때 `const T&`를 쓰는 것은 현대 C++의 기본형이다. 이 형식을 외워 두면 함수 시그니처만 보고도 "복사 없이 읽기만 하는구나", "수정 가능한 참조구나", "작은 값 타입이라 값 전달이구나"를 빠르게 해석할 수 있다.

10.5 Optional 반환

실패 가능성을 타입으로 드러내기

```
std::optional<User> findUser(const std::string &name) {
    auto it = db.find(name);
    if (it != db.end())
        return it->second;
    return std::nullopt;
}
```

optional이 좋은 이유는 실패 가능성을 문서가 아니라 타입에 넣기 때문이다. 함수 이름만 보고는 실패 가능성을 놓칠 수 있지만, 반환 타입이 optional<T>이면 호출자는 반드시 "값이 없을 수도 있다"는 사실을 처리해야 한다. Python의 None보다 더 강한 신호라고 보면 된다.

10.6 Virtual Interface

인터페이스 정의하고 자식이 구현

```
class Serializer {
public:
    virtual ~Serializer() = default;
    virtual std::vector<uint8_t> serialize(const Data &data) = 0;
    virtual Data deserialize(const std::vector<uint8_t> &raw) = 0;
    virtual std::string name() const { return "unknown"; }
};

class JsonSerializer : public Serializer {
public:
    std::vector<uint8_t> serialize(const Data &data) override;
    Data deserialize(const std::vector<uint8_t> &raw) override;
    std::string name() const override { return "json"; }
};
```

이 패턴은 런타임에 구현체가 바뀔 수 있을 때의 표준형이다. 함께 외워야 할 관용구는 virtual ~Base() = default;와 override다. 전자는 다형적 삭제를 안전하게 만들고, 후자는 재정의 실수를 컴파일 타임에 잡는다.

10.7 string_view / StringRef

복사 없이 문자열 빌려보기

```
void log(std::string_view msg) {
    std::cout << msg << "\n";
}
```

주의

`string_view`는 소유하지 않는다. 원본 문자열보다 오래 살면 dangling view가 된다.

이 패턴은 성능 때문에 중요하지만, 그보다 더 중요한 것은 **소유권을 명확히 드러내는 인터페이스**를 만들 수 있다는 점이다. 함수가 `std::string`을 받으면 "혹시 복사해서 소유하나?"를 의심하게 되지만, `string_view`는 "나는 읽기만 한다"는 의도를 강하게 드러낸다. LLVM의 `StringRef`도 같은 철학이다.

10.8 Template 함수/클래스

타입을 매개변수로 받는 코드

```
template <typename T>
T maxElement(const std::vector<T> &items) {
    T result = items[0];
    for (const auto &item : items) {
        if (item > result) result = item;
    }
    return result;
}

template <typename T>
class Stack {
    std::vector<T> data;
public:
    void push(const T &val) { data.push_back(val); }
    T pop() {
        T top = std::move(data.back());
        data.pop_back();
        return top;
    }
};
```

템플릿은 MLIR과 현대 C++에서 피할 수 없는 문법이다. 함수 템플릿은 "호출 시 타입 추론", 클래스 템플릿은 "선언 시 타입 명시"라는 두 가지만 정확히 익혀 두면, 상당수의 패턴이 읽히기 시작한다.

10.9 constexpr + static_assert

컴파일 타임 계산과 검증

```
constexpr int factorial(int n) {
    return (n <= 1) ? 1 : n * factorial(n - 1);
}

struct PacketHeader {
    uint32_t size;
```

```
    uint16_t type;
};

static_assert(std::is_trivially_copyable_v<PacketHeader>,
              "PacketHeader must be trivially copyable");
```

std::is_same_v가 뜻하는 것

`std::is_same_v<A, B>`는 "A와 B가 완전히 같은 타입인가"를 나타내는 컴파일 타임 bool 값이다. 함수 호출이 아니라 타입에 대한 질문이다.

이 섹션은 템플릿 코드가 왜 난해해 보이는지를 푸는 열쇠이기도 하다. 템플릿 메타프로그래밍은 결국 "타입에 대한 질문을 코드 안에 넣는 법"이다. `std::is_same_v`, `std::is_integral_v`, `static_assert` 같은 도구를 이해하면, 적어도 남이 쓴 현대 C++ 템플릿 코드를 해석할 수 있게 된다.

10.10 Aggregate 초기화

struct를 한 번에 초기화

```
struct Config {
    int width = 640;
    int height = 480;
    bool fullscreen = false;
};

Config c1;
Config c2{1920, 1080};
Config c3{.width = 1920, .height = 1080, .fullscreen = true};
```

단순 데이터 묶음은 생성자를 과하게 만들기보다 aggregate 초기화를 활용하는 편이 읽기 쉽다. 기본값, 위치 기반 초기화, C++20 designated initializer까지 외워 두면 설정 객체를 다룰 때 매우 편하다.

10.11 Scope Guard와 Custom Deleter

Scope Guard의 최소 형태

```
struct Resource {
    int handle;
};

Resource *acquire();
void release(Resource *r);

struct ScopeGuard {
    std::function<void()> fn;
    ~ScopeGuard() { fn(); }
```

```
};

void process() {
    Resource *resource = acquire();
    ScopeGuard guard{[&]() { release(resource); }};
    // 중간에 하거나return 예외가 나도 release 실행
}
```

unique_ptr + custom deleter

```
auto resource = std::unique_ptr<Resource, decltype(&release)>(
    acquire(), &release
);
```

배운 것 정리

- cleanup을 일으키는 것은 함수 포인터 자체가 아니라 **관리 객체의 소멸자**다.
- `decltype(\&release)`는 `release` 함수 포인터의 타입을 자동으로 가져온다.
- `unique_ptr`는 "자원 + 해제 방법"을 같이 들고 있는 RAII 객체다.

Scope Guard를 이해하면 "왜 스코프를 나가면 cleanup이 자동으로 되지?"라는 질문이 풀린다. 답은 간단하다. 스코프를 나갈 때 **객체가 소멸하고**, 그 객체의 **소멸자 안에 cleanup 호출 코드가 있기 때문**이다. 함수 포인터는 그 cleanup이 무엇인지 저장하는 정보일 뿐이다.

10.12 CRTP

CRTP의 기본 모양

```
template <typename Derived>
class Printable {
public:
    void print() const {
        const auto &self = static_cast<const Derived &>(*this);
        std::cout << self.toString() << "\n";
    }
};

class Dog : public Printable<Dog> {
public:
    std::string toString() const { return "Dog"; }
};
```

왜 쓰는가

CRTP는 부모가 자식 타입을 템플릿 인자로 받아서, virtual 없이 자식 메서드에 접근하는 패턴이다. MLIR의 `PassWrapper<MyPass, OperationPass<ModuleOp>>`가 대표적인 예다.

처음 보면 상속처럼 보이지만, 사실은 "부모가 자식을 컴파일 타임에 아는 템플릿 패턴"이다. 그래서 런타임 다형성을 제공하는 virtual과 달리, 정적 다형성을 제공한다. MLIR에서 CRTP가 많은 이유는 프레임워크 공통 로직은 부모에 두되, 자식 타입 정보는 컴파일 타임에 확보하고 싶기 때문이다.

10.13 enum class + switch

안전한 열거형

```cpp
enum class Color { Red, Green, Blue };

std::string colorName(Color c) {
    switch (c) {
    case Color::Red:   return "red";
    case Color::Green: return "green";
    case Color::Blue:  return "blue";
    }
    llvm_unreachable("unknown color");
}
```

`enum class`는 값 영역을 제한하면서 이름 충돌을 줄인다. C++ 코드에서 상태 종류나 모드를 표현할 때 string 상수보다 훨씬 안전하다. switch와 함께 쓰면 빠뜨린 case를 컴파일러 경고로 잡을 수 있다는 점도 크다.

10.14 structured bindings

pair / tuple / struct 분해

```cpp
std::unordered_map<std::string, int> scores = {{"alice", 95}};
for (const auto &[name, score] : scores) {
    std::cout << name << ": " << score << "\n";
}
```

이 패턴은 단순한 문법 설탕 같아 보이지만, 실제로는 코드의 의도를 훨씬 잘 드러낸다. `first`, `second` 대신 의미 있는 이름으로 바로 분해해서 읽을 수 있기 때문이다. MLIR에서도 결과 쌍이나 map 순회에서 자주 유용하다.

10.15 콜백과 std::function

함수를 값처럼 다루기

```
class EventHandler {
    std::function<void(int)> callback;
public:
    void setCallback(std::function<void(int)> fn) {
        callback = std::move(fn);
    }
};
```

std::function은 함수 포인터보다 무겁지만, 람다와 함수 객체를 통일된 타입으로 저장할 수 있다. 바로 호출만 할 때는 템플릿이 더 낫고, 저장해 뒀다가 나중에 호출해야 할 때는 std::function이 편하다.

10.16 이동으로 소유권 넘기기

push_back, move, emplace_back

```
std::vector<std::string> names;

std::string name = "alice";
names.push_back(name);            // 복사
names.push_back(std::move(name)); // 이동
names.emplace_back("bob");        // 컨테이너 안에서 직접 생성
```

push_back(name), push_back(std::move(name)), emplace_back(...)는 각각 다른 의도를 가진다. 원본을 유지할지, 소유권을 넘길지, 그 자리에서 바로 만들지를 구분하는 감각을 아예 패턴으로 외워 두는 편이 좋다.

10.17 반복자 무효화 방지

erase가 다음 반복자를 반환한다

```
for (auto it = vec.begin(); it != vec.end(); ) {
    if (bad(*it))
        it = vec.erase(it);
    else
        ++it;
}
```

이 함정은 Python에서도 비슷하게 있지만, C++에서는 iterator invalidation이라는 더 직접적인 버그로 터진다. 컨테이너를 순회하면서 수정할 때는 "지금 들고 있는 반복자가 여전히 유효한가"를 항상 함께 생각해야 한다.

Chapter 11

자주 밟는 함정

11.1 Dangling reference / dangling view

참조나 `string_view`는 소유하지 않기 때문에 원본보다 오래 살면 즉시 위험해진다. 이 문제는 컴파일러가 모두 잡아주지 못하므로, 인터페이스를 설계할 때부터 "이 값이 원본보다 오래 살 수 있는가"를 질문하는 습관이 필요하다.

11.2 Object slicing

부모 타입 값으로 자식 객체를 복사하면 자식 부분이 잘린다. 다형성이 필요하면 값 복사가 아니라 포인터/참조를 써야 한다. 이 문제는 virtual을 쓴다고 자동 해결되지 않는다.

11.3 멤버 초기화 순서

생성자 초기화 리스트의 순서가 아니라 **멤버 선언 순서대로** 초기화된다. 그래서 초기화 리스트를 보기 전에 클래스 멤버 선언 순서를 먼저 봐야 한다.

11.4 Implicit conversion

한 인자짜리 생성자에 `explicit`이 없으면 의도치 않은 암시적 변환이 일어날 수 있다. 값 래퍼 타입이나 작은 설정 타입에서 특히 자주 보이는 함정이다.

11.5 헤더와 구현 분리

일반 함수/클래스는 선언과 구현을 나누되, 템플릿은 대개 정의까지 헤더에 있어야 한다. 이 규칙을 모르면 "컴파일은 되는데 링크가 깨지는" 상황을 반복해서 만나게 된다.

Chapter 12

참고용 완전한 형태

12.1 값 타입 클래스

데이터를 담는 정석적 클래스

```
class Buffer {
public:
    explicit Buffer(size_t size) : data_(size, 0) {}

    size_t size() const { return data_.size(); }
    uint8_t *data() { return data_.data(); }
    const uint8_t *data() const { return data_.data(); }

private:
    std::vector<uint8_t> data_;
};
```

값 타입 클래스의 표준형을 하나 외워 두면, “자원은 멤버가 관리하고 인터페이스는 작게 노출한다”는 감각을 빠르게 익힐 수 있다.

12.2 Interface + 구현 + Factory

상속 계층의 표준형

```
class Animal {
public:
    virtual ~Animal() = default;
    virtual std::string speak() const = 0;
};

class Dog : public Animal {
public:
    std::string speak() const override { return "woof"; }
};
```

```

std::unique_ptr<Animal> createAnimal(bool barking) {
    if (barking) return std::make_unique<Dog>();
    return nullptr;
}
```

이 구조는 interface, 구현, 생성 분기를 분리하는 가장 표준적인 모양이다. 다형성이 필요한 코드를 설계할 때 거의 기계적으로 떠올릴 수 있을 정도로 익혀 두는 편이 좋다.

12.3 한 번에 외울 패턴 묶음

상황	먼저 떠올릴 것	대표 패턴
자원 정리	소멸자	RAII, Scope Guard, unique_ptr
런타임 다형성	부모 인터페이스	virtual + factory
컴파일 타임 분기	타입 정보	template, constexpr, type traits
복사 줄이기	view / 참조	const ref, string_view, StringRef
공통 로직 재사용	정적 다형성	CRTP

Part IV

MLIR을 위한 C++

Chapter 13

MLIR 큰 그림

MLIR은 "IR을 하나만 두는 시스템"이 아니라 **여러 dialect와 여러 단계의 lowering을 조합하는 인프라**다.

이 파트에서 얻어야 할 감각

MLIR을 문법으로 배우면 금방 길을 잃는다. 먼저 "어떤 추상화를 어디 단계에서 쓰는가", "왜 TableGen과 C++가 나뉘는가", "왜 pass와 pattern이 분리되는가"를 이해해야 한다.

13.1 핵심 개념

개념	의미
Dialect	연산과 타입의 집합. 예: `func`, `arith`, `linalg`
Operation	IR의 기본 단위. 결과 값, 속성, 피연산자, region을 가질 수 있다
Pass	IR을 읽고 변형하는 단계
Pattern Rewriting	특정 op를 다른 op들로 치환하는 규칙 기반 변환
TableGen	op, type, attr, pass 선언을 기술하는 메타 언어

왜 MLIR이 C++과 잘 맞는가

MLIR은 타입이 풍부하고, 템플릿과 CRTP를 많이 활용하며, 수많은 작은 객체의 수명과 성능을 정교하게 다룬다. 그래서 C++ 기본기가 곧 MLIR 생산성으로 이어진다.

LLVM IR와 비교해 보면 MLIR의 특징은 "중간 표현이 하나로 고정되어 있지 않다"는 점이다. 문제 도메인마다 dialect를 만들고, 점진적으로 더 낮은 수준으로 변환하면서 의미를 잃지 않으려 한다. 그래서 MLIR을 이해하려면 단일 IR 문법을 외우는 대신, **변환 파이프라인 전체를 보는 습관**이 필요하다.

13.2 IR 계층을 어떻게 상상하면 좋은가

가장 큰 단위는 `ModuleOp`이고, 그 안에 함수 같은 연산이 들어가고, 함수 안에는 block과 operation이 들어간다. 즉 MLIR의 많은 것은 "모든 것이 operation이다"라는 발상 위에 서 있다. 함수도 operation이고, 반복문도 operation이고, 사용자 정의 dialect의 연산도 operation이다.

이 점을 이해하면 왜 builder, verifier, rewrite pattern, pass가 모두 operation 중심 API를 가지는지 자연스럽게 보인다. MLIR은 AST를 직접 고치는 느낌보다, operation 그래프를 재작성하는 느낌에 가깝다.

13.3 대표 작업 흐름

MLIR 개발의 반복 루프

```
TableGen 수정
-> generated headers / cpp 재생성
-> C++ dialect / pass 구현 수정
-> 빌드
-> mlir-opt 또는 내 툴로 테스트
```

실제로 작업할 때는 이 루프를 수십 번 반복한다. 따라서 중요한 것은 "한 번 정확히 짜기"보다 "어디를 고치면 어떤 generated 코드가 바뀌고, 어떤 pass에서 실패하는지 빠르게 좁히는 능력"이다.

Chapter 14

TableGen과 코드 생성

14.1 TableGen은 왜 필요한가

MLIR 코드는 반복이 많다. op 이름, operands, results, traits, builders, assembly format 같은 정보는 선언적으로 적고, 나머지는 생성 코드에 맡기는 편이 낫다.

개념적 예시: TableGen이 표현하는 것

```
def My_AddOp : My_Op<"add", [Pure]> {
  let arguments = (ins I32:$lhs, I32:$rhs);
  let results = (outs I32:$result);
  let assemblyFormat = "$lhs `,` $rhs attr-dict `:` type($result)";
}
```

배운 것 정리

- TableGen은 사람이 반복 선언을 쓰고
- 생성 코드는 builder, accessor, parser/printer 보일러플레이트를 담당한다
- hand-written C++는 검증 로직, 리라이트 로직, 특수한 동작에 집중한다

TableGen을 처음 볼 때는 "왜 그냥 C++로 안 쓰지?"라는 의문이 생긴다. 답은 반복 때문이다. op 하나를 손으로 모두 쓰기 시작하면 이름, operand/result accessor, builder, parser, printer, trait 연결 코드가 끝없이 반복된다. TableGen은 이 반복을 선언적으로 압축한다.

14.2 TableGen과 hand-written C++의 경계

좋은 기준은 이렇다. **선언적인 구조**는 TableGen으로, **행동 로직**은 C++로 둔다. op의 형태, 속성, 조립 문법은 TableGen 쪽에 가깝고, 복잡한 검증 규칙, 특수 lowering, 알고리즘성 로직은 C++ 쪽에 가깝다.

이 경계를 이해하면 generated 파일에서 에러가 나더라도 원인을 추적하기 쉬워진다. generated 코드는 보통 증상이고, 원인은 원래의 `.td` 선언이거나 그 선언과 손으로 쓴 C++의 불일치다.

14.3 실전 감각

- op 정의를 바꾸면 generated 파일이 함께 바뀐다.
- 에러가 generated 파일에서 보이더라도 원인은 원래의 `.td` 선언인 경우가 많다.
- 빌드 에러를 읽을 때는 "내가 선언한 op 시그니처가 생성 코드와 맞는가"를 먼저 본다.

Chapter 15

MLIR에서 반복되는 설계 패턴

15.1 Builder 패턴

OpBuilder는 상태를 가진 생성기

```
auto convOp = builder.create<MyConvOp>(
    loc, resultType, input, weight, bias
);
```

`OpBuilder`의 핵심은 단순한 factory가 아니라 **삽입 위치 상태를 가진 builder**라는 점이다. 그래서 `setInsertionPoint...()`와 `create<Op>()`의 순서가 중요하다. 이 감각이 없으면 "왜 op가 엉뚱한 위치에 생기지?" 같은 버그를 이해하기 어렵다.

15.2 Visitor / Dispatch 패턴

MLIR 코드를 읽다 보면 "여러 종류의 op를 종류별로 다르게 처리"하는 코드가 계속 나온다. 어떤 경우에는 if-else나 switch처럼 직접 분기하고, 어떤 경우에는 패턴 등록을 통해 프레임워크가 대신 분기한다. 둘 다 본질은 dispatch 패턴이다.

15.3 Registry 패턴

패턴 등록

```
RewritePatternSet patterns(ctx);
patterns.add<MyOpLowering>(ctx);
patterns.add<OtherOpLowering>(ctx);
```

직접 if-else를 쓰는 대신, 처리기를 등록해 두고 프레임워크가 자동 매칭하게 하는 방식이다. MLIR의 rewrite framework가 강력한 이유는 새 op를 추가할 때 기존 거대한 분기문을 건드리지 않고도 확장할 수 있기 때문이다.

15.4 Template Method 패턴

`OpConversionPattern<T>`의 자식 클래스는 전체 알고리즘을 다시 쓰지 않는다. 프레임워크가 매칭, 상태 관리, 실패 전파 같은 공통 구조를 제공하고, 우리는 `matchAndRewrite()`만 구현한다. 이것이 Template Method 패턴의 실질적 이점이다.

Chapter 16

MLIR 프로젝트의 CMake 구조

16.1 최소 골격

MLIR 도구 프로젝트의 최소 뼈대

```
cmake_minimum_required(VERSION 3.20)
project(MyMlirProject LANGUAGES CXX)

find_package(LLVM REQUIRED CONFIG)
find_package(MLIR REQUIRED CONFIG)

list(APPEND CMAKE_MODULE_PATH "${LLVM_CMAKE_DIR}")
list(APPEND CMAKE_MODULE_PATH "${MLIR_CMAKE_DIR}")
include(AddLLVM)
include(AddMLIR)

add_subdirectory(lib)
add_subdirectory(tools)
```

도구 타겟 예시

```
add_executable(my-opt my-opt.cpp)
target_link_libraries(my-opt
  PRIVATE
    MyDialect
    MyConversion
    MLIRIR
    MLIRParser
    MLIRPass
)
```

16.2 패스 개발 흐름

Lowering 패턴의 전형적인 모양

```
class MyOpLowering : public OpConversionPattern<MyOp> {
public:
    using OpConversionPattern<MyOp>::OpConversionPattern;

    LogicalResult matchAndRewrite(
        MyOp op,
        OpAdaptor adaptor,
        ConversionPatternRewriter &rewriter) const override {
        rewriter.replaceOpWithNewOp<arith::AddIOp>(
            op, adaptor.getLhs(), adaptor.getRhs());
        return success();
    }
};
```

이 패턴의 의미를 코드 한 줄씩 해석해 보면 다음과 같다. `MyOp`는 "어떤 op를 처리하는지"를 컴파일 타임에 고정한다. `matchAndRewrite`는 실제 변환 로직을 담는다. `OpAdaptor`는 원래 op의 피연산자를 변환된 타입 관점에서 접근하게 해준다. `ConversionPatternRewriter`는 새 op를 만들고 기존 op를 교체하는 도구다.

PassWrapper를 이용한 패스 껍데기

```
struct LowerMyToArithPass
    : public PassWrapper<LowerMyToArithPass, OperationPass<ModuleOp>> {
    void runOnOperation() override {
        RewritePatternSet patterns(&getContext());
        patterns.add<MyOpLowering>(&getContext());
        // conversion setup ...
    }
};
```

패턴과 패스는 역할이 다르다. 패턴은 "개별 op를 어떻게 바꿀지"를 정의하고, 패스는 "어떤 패턴들을 어떤 목표(target) 아래 어떤 순서/정책으로 적용할지"를 정의한다. 이 둘을 분리해서 생각해야 MLIR 코드 구조가 보인다.

여기서 C++ 개념이 어떻게 연결되는가

- `OpConversionPattern<MyOp>`는 template다.
- `PassWrapper<Derived, Base>`는 CRTP다.
- `runOnOperation()`은 virtual override다.
- `RewritePatternSet`과 각종 IR 객체는 수명 관리가 중요하므로 참조와 포인터 감각이 필요하다.

16.3 새 dialect / op / pass를 추가할 때의 사고 순서

1. 먼저 이 개념이 새 dialect가 필요한지, 기존 dialect 안의 새 op면 충분한지 결정한다.
2. op의 모양은 TableGen에서 선언하고, verifier나 복잡한 동작은 C++에 둔다.
3. lowering이 필요하면 개별 `OpConversionPattern`을 만든다.

4. 그 패턴들을 pass 안에서 등록하고, target을 정의한다.
5. 마지막으로 `mlir-opt`나 프로젝트 도구에서 실제 파이프라인에 연결한다.

이 순서를 머릿속에 갖고 있으면, 처음 보는 MLIR 저장소에서도 "아, 이 프로젝트는 어디에 어떤 책임을 두었구나"를 훨씬 빨리 읽을 수 있다.

16.4 MLIR 학습 체크리스트

실전 체크리스트

- dialect, operation, pass, pattern의 역할을 구분할 수 있는가
- `mlir-opt`로 패스를 실행해 볼 수 있는가
- TableGen 선언과 generated 코드의 관계를 설명할 수 있는가
- CMake에서 MLIR 라이브러리를 링크하는 최소 구성을 쓸 수 있는가
- 패턴 하나와 패스 하나를 새로 추가하는 흐름을 머릿속에서 그릴 수 있는가

Chapter 17

빠르게 익숙해지기 위한 최종 암기표

배운 것 정리

- CMake는 타겟 그래프를 선언하는 도구다.
- C++를 읽을 때는 문법보다 수명, 소유권, 디스패치 방식을 먼저 본다.
- 실전에서 가장 자주 보는 패턴은 RAII, unique_ptr, const ref, optional, factory, virtual, CRTP다.
- MLIR 코드는 template, CRTP, virtual, generated code가 한 코드베이스에서 동시에 섞여 나온다.
- 막힐 때는 "이 객체는 누가 소유하는가", "이 호출은 런타임인가 컴파일 타임인가", "이 코드는 generated인가 hand-written인가"를 먼저 물어본다.

Quiz

Q1. `unique_ptr<Resource, decltype(\&release)>`에서 cleanup을 실제로 일으키는 것은 무엇인가?

A. 함수 포인터의 소멸이 아니라 `unique_ptr` 소멸자이다.

Q2. `PassWrapper<MyPass, OperationPass<ModuleOp>>`는 왜 CRTP인가?

A. 부모 템플릿이 자식 타입 `MyPass`를 인자로 받아 compile-time에 자식 타입을 알기 때문이다.

Q3. MLIR CMake에서 가장 먼저 확인할 외부 의존성 선언은 무엇인가?

A. 보통 `find_package(LLVM REQUIRED CONFIG)`와 `find_package(MLIR REQUIRED CONFIG)`다.

다음 단계

이 책을 한 번 읽고 끝내지 말고, 실제 코드에서 반복해서 대응시키는 것이 중요하다.

1. `mlir-opt`나 프로젝트 전용 도구의 `main()`을 읽어 본다.
2. dialect 하나의 TableGen 정의와 생성 결과를 연결해서 본다.
3. lowering 패턴 하나를 처음부터 끝까지 추적한다.
4. CMake에서 타겟 하나가 어떤 라이브러리를 타고 들어가는지 따라가 본다.

이 과정을 세 번 정도 반복하면, C++ 문법이 아니라 코드 구조가 먼저 보이기 시작한다.